

GRUPO SANVEST · TRANSFORMACIÓN DIGITAL

Sanvest BI

Documentación del Proyecto

Reportes · Inteligencia de Negocios

Plataforma web que reemplaza el modelo de Power BI del directorio por dashboards en vivo, reconciliados 1:1, con carga de informes y control de accesos.

FastAPI

React · Vite · Recharts

PostgreSQL

7 unidades de negocio

Login por rol

Documento técnico + Manual de usuario

Versión 27-jul-2026 · Preparado para Grupo Sanvest

Índice

Dos partes: la **Parte I** es técnica (para quien opera o hereda el sistema); la **Parte II** es el manual para quien usa la aplicación.

Parte I — Documentación técnica

- 1 Visión general y propósito
- 2 Arquitectura y stack
- 3 Estructura del repositorio
- 4 Las 7 unidades de negocio
- 5 Puesta en marcha (local)
- 6 Producción y despliegue (VPS)
- 7 Pipeline de datos / ETL
- 8 Autenticación, roles y bitácora
- 9 Cómo agregar o actualizar una unidad

- 10 Convenciones y reglas críticas

Parte II — Manual de usuario

- A Ingreso a la aplicación
- B Menú principal
- C Cómo leer los dashboards
- D Recorrido por unidad
- E Glosario de indicadores
- F Funciones: clave, comentarios, PDF, PPT
- G Para administradores

En una frase: Sanvest BI toma los Excel/informes que hoy alimentan al directorio, los procesa con un ETL en Python, los guarda en PostgreSQL y los muestra como dashboards web interactivos — reemplazando al archivo de Power BI, con acceso por usuario y rol.

PARTE I

Documentación técnica

1 · Visión general y propósito

Sanvest BI es una **aplicación web de reportería financiera** para el Grupo Sanvest (grupo inmobiliario chileno). Nació para migrar el modelo de **Power BI** del directorio (`Sanvest BI 24.0122026.pbix`) a una web propia, **reconciliada 1:1** contra ese archivo, y luego creció con funciones que el .pbix no tenía: carga de informes por la misma web, login por usuario, perfilado por unidad, bitácora de accesos, comentarios del directorio y auditoría de datos.

La usan los **directores y el equipo de finanzas**. Cada usuario ve solo las unidades de negocio que le corresponden. El idioma de trabajo del proyecto es **español**.

2 · Arquitectura y stack

Backend — API

FastAPI + SQLAlchemy (Python). Expone endpoints REST que leen la base y entregan JSON al front. Cero dependencias exóticas: el propio proyecto trae su loader de `.env`. Punto de entrada: `api/main.py`.

Frontend — Web

React + Vite + TypeScript, gráficos con **Recharts**. Se compila a estáticos (`frontend/dist` , versionado en git). Configuración declarativa por unidad.

Base de datos

PostgreSQL en producción (base `sanvest` en el VPS) y **SQLite** para desarrollo local (`db/sanvest_bi_dev.sqlite`). El mismo código sirve a ambas (SQL portable).

ETL

Módulos `etl/connect_*.py` con funciones `apply_*` / `build_*` que leen los Excel/informes y escriben las tablas. Un **catálogo por unidad** en `api/catalog/*.json` describe qué tablas y campos usa cada dashboard.

Flujo de datos, de punta a punta: Excel/informe → `apply_*` (ETL) → tabla en PostgreSQL → endpoint FastAPI → dashboard React. Lo que ve el usuario en la web es **siempre** lo que está en la base de producción.

3 · Estructura del repositorio

Carpeta / archivo	Qué contiene
api/	Backend FastAPI. <code>main.py</code> (endpoints), <code>auth.py</code> (login/roles/bitácora/comentarios), <code>db.py</code> (engine), <code>filetables.py</code> (tablas servidas desde JSON empaquetado), <code>catalog/*.json</code> (config por unidad), <code>data/</code> (datos estáticos), <code>specs/</code> .
etl/	Conectores y transformaciones: <code>connect_*.py</code> , <code>statement_extractor.py</code> (extracción declarativa de Balance/EERR), <code>specs/*.json</code> , <code>db.py</code> (<code>get_engine()</code>), <code>audit.py</code> .
frontend/	<code>src/pages/*Dashboard.tsx</code> (una página por unidad), <code>src/components/</code> (BalanceSheet, PnLMatrix, charts, KpiCard, Gauge...), <code>src/auth.tsx</code> , <code>dist/</code> (build compilado, versionado).
scripts/	Scripts operativos reutilizables: <code>manage_users.py</code> (usuarios), y utilidades de carga/migración a prod (patrón <code>.venv/Scripts/python</code>).
docs/	Documentación técnica: mapeos ETL, reconciliaciones por unidad, <code>deploy.md</code> , <code>manual_carga.md</code> , decisiones de diseño.
db/	SQLite de desarrollo, DDL y respaldos (<code>db/backups/</code>).
.env	Configuración sensible (credenciales PostgreSQL <code>PG*</code> , secreto de tokens). No se versiona.

4 · Las 7 unidades de negocio

Cada unidad es una página de dashboard con su propio color de marca, catálogo y ETL. El acceso se perfila por unidad (un *viewer* ve solo las suyas; el *admin* ve todas).

DV — Desarrollo para la Venta

INMOBILIARIO · MILLALONGO, STA. VICTORIA 99, STA. VICTORIA 155

Avance de ventas y unidades vendidas (fuente: Estadística de Ventas), evolución de costos, deuda por proyecto y **capital de socios** (Egresos – Deuda – Preventas), con amortizado manual. ETL: `connect_dv.py` (`apply_dv`).

RR — Renta Residencial (LAR Group)

MULTIFAMILY · HOLDING LAR, SOHO, PARK

Informe de gestión (P&L), ocupación mes y YTD, KPIs por edificio. El cuadro de KPIs por edificio se recalcula **en vivo** desde la base operativa `SQLLAR` (`connect_sqllar.py`). ETL principal: `connect_lar.py` .

Hotel — OLÁ Hotel

HOTELERÍA · OLÁ PROVIDENCIA

Indicadores financieros del hotel, ocupación mes y YTD, deuda. ETL: `connect_hotel.py` (`apply_ccpp` con guard anti-regresión para no pisar meses ya cerrados).

USA — Estados Unidos

BEMISTON · MILA · ST GRAND

P&L y KPIs de las propiedades en EE.UU. Los combos salen de los *Operating Statements* (`usa_pnl`); el YTD se acumula del Real. ETL: `connect_usa.py` (`apply_usa_budget` , `apply_usa_kpis`) con detección automática del formato del reporte.

ICEMM — Construcción

INDICADORES FINANCIEROS · FLUJO DE CAJA

Informe de gestión de la constructora (FY / YTG / YTD), flujo de caja colapsable. ETL: `connect_icemm.py` . Las proyecciones (FY/YTG) dependen del archivo REV que se sube. (El control de *Obras* vive en otra base del mismo Postgres, pendiente de conectar.)

Atémpora — Civitas

MULTI-USO · OFICINAS, LOCALES, ARRIENDOS, MOROSIDAD

Informe de gestión (EERR) desde el Flujo de Caja, KPIs de comercialización, ocupación del edificio, cuadro de arriendos, ventas y morosidad por tramos. ETL: `connect_atempora.py` (`apply_atempora` , `_kpis` , `_morosidad`).

Grupo — Estados Financieros

GRUPO SANVEST · BALANCE · EERR (CONSOLIDADO DEL DIRECTORIO)

Balance jerárquico (réplica exacta del Excel, con notas al hover) y Estado de Resultados. Se carga por trimestre. ETL declarativo: `connect_grupo.py` + `statement_extractor.py` guiado por `etl/specs/Grupo.{balance,eerr}.json` , con validación de cuadro.

5 · Puesta en marcha (local)

Componente	Comando / ruta
API (backend)	<code>.venv/Scripts/python -m uvicorn api.main:app --port 8077 --host 127.0.0.1</code>
Frontend (dev)	<code>cd frontend && npm run dev</code> → Vite en <code>:5176</code>
Frontend (build)	<code>cd frontend && npm run build</code> → genera <code>frontend/dist</code>
Base dev	SQLite <code>db/sanvest_bi_dev.sqlite</code> (para probar ETL sin tocar prod)

Ojo: la API en `:8077`, tal como está configurada con las variables `PG*` del `.env`, lee **PRODUCCIÓN**. Lo que se ve en la app corriendo local es la base de prod. El SQLite dev solo se usa apuntando el ETL a él explícitamente. Los puertos `:8000` y `:5173` son de otro proyecto — este usa `:8077` y `:5176`.

6 · Producción y despliegue (VPS)

Producción es un **PostgreSQL** (base `sanvest`) en un VPS. La conexión se configura con variables `PG*` en el `.env` (host, base, usuario, clave). El identificador de columnas usa comillas dobles (`qi()`), porque muchos nombres traen espacios/mayúsculas del Excel de origen.

Ciclo de despliegue típico

1. `git pull` en el VPS para traer los cambios.
2. **Reiniciar la API** (uvicorn desde el `.venv`) — necesario para que tome cambios de backend y para que aparezcan tablas nuevas.
3. Servir el bundle `frontend/dist` (ya viene compilado y versionado).
4. En el navegador, `Ctrl` + `F5` para saltar caché.

Cambios que NO requieren desplegar: los cambios de *datos* (correcciones directas en la base de prod) se ven con solo recargar la página. Solo los cambios de *código* (backend/front) requieren pull + reinicio + build.

Privilegios en prod: el usuario de base de datos de producción **sí** tiene privilegio `CREATE` — se pueden crear tablas y columnas y escribir directo a prod. (Esto corrige una nota antigua que decía lo contrario.) Aun así, toda escritura a prod se hace con criterio: script reutilizable, *dry-run* primero, y respaldo cuando corresponde.

La guía de endurecimiento (nginx + TLS, secreto de tokens, API como servicio systemd para que no se caiga al cerrar sesión) está en `docs/deploy.md`.

7 · Pipeline de datos / ETL

La carga de cada unidad vive en su `connect_*.py` con una función `apply_*`. Todas siguen el mismo patrón **seguro**: leen el Excel/informe, normalizan, y escriben la tabla con **DELETE + INSERT** del segmento afectado (nunca *DROP*), de modo que la carga es idempotente y no rompe el resto de los datos.

Función <code>apply_*</code>	Unidad / propósito
<code>apply_dv</code>	DV — ventas, costos, deuda, capital socios
<code>apply_informes</code> , <code>apply_yardi</code>	RR (LAR) — P&L y KPIs
<code>apply_ccpp</code>	Hotel (OLÁ) — con guard anti-regresión
<code>apply_usa_budget</code> , <code>apply_usa_kpis</code>	USA — P&L y KPIs
<code>apply_icemm</code>	ICEMM — informe de gestión + flujo
<code>apply_atempora</code> , <code>apply_atempora_kpis</code> , <code>apply_atempora_morosidad</code>	Atémpora (Civitas)
<code>apply_grupo</code> / <code>build_grupo</code>	Grupo — Balance + EERR por trimestre
<code>apply_deuda</code>	Cronograma de deuda (activos)

Piezas transversales

- **Catálogos** (`api/catalog/*.json`): declaran qué tablas y campos consume cada dashboard. Cambiar un dashboard es, muchas veces, editar su catálogo — sin tocar código.
- **Extractor declarativo** (`statement_extractor.py` + specs): para el Grupo, lee Balance y EERR desde el Excel guiado por `etl/specs/Grupo/*.json` (niveles, notas, columnas de valor), y valida el cuadro antes de escribir.
- **Tablas desde archivo** (`api/filetables.py`): datos estáticos (p.ej. el cronograma de deuda) se sirven desde un JSON empaquetado si la tabla no existe en la base.
- **Auditoría** (`etl/audit.py`): chequeos deterministas (datos viejos, huecos, valores pegados, tablas vacías) expuestos en el panel de administración.

Carga por la web: el admin sube los informes desde la propia app (panel de carga). El endpoint `/units/{unit}/upload-informes` despacha el archivo al `apply_*` correcto según el nombre del archivo.

8 · Autenticación, roles y bitácora

Todo el sistema de acceso vive en `api/auth.py`, sin dependencias nuevas (hashing y tokens con la librería estándar de Python).

Concepto	Detalle
Usuario	El username es el email de la persona. Tabla <code>app_users</code> .
Contraseña	Hash <code>pbkdf2_hmac</code> (SHA-256, 200k iteraciones) con salt por usuario; comparación en tiempo constante. Nunca se guarda en claro.
Token	Mini-JWT firmado con HMAC-SHA256 (secreto <code>SANVEST_AUTH_SECRET</code>), vigencia 12 h. El front lo manda como <i>Bearer</i> en cada llamada.
Roles	admin — ve todas las unidades y puede cargar datos · viewer — ve solo su lista de unidades, solo lectura.
Bitácora	<code>access_log</code> registra quién entra y a qué unidades (panel «Accesos»). Es <i>best-effort</i> : un fallo al registrar nunca bloquea el login.
Comentarios	Foro por unidad (tabla <code>comments</code>) para el directorio.

Login case-insensitive (jul-2026): el email se normaliza a **minúsculas y sin espacios** en todas las funciones que lo usan como clave, así `Foo@Bar.CL` y `foo@bar.cl` entran igual. El campo del login además evita que el teclado del móvil capitalice la primera letra.

Gestión de usuarios

Desde la web (panel Admin ► Usuarios) o por consola con `scripts/manage_users.py`:

- `python scripts/manage_users.py create --admin sleyton@sanvest.cl --full-name "S. Leyton"`
- `python scripts/manage_users.py create jperez@sanvest.cl --units RR,USA (viewer)`
- `list · passwd <user> · set-units · set-role · deactivate / activate`

9 · Cómo agregar o actualizar una unidad

- ETL:** escribir/ajustar el `apply_*` en `etl/connect_*.py` que lee el Excel y escribe la(s) tabla(s) con DELETE+INSERT.
- Catálogo:** declarar tablas y campos en `api/catalog/<Unidad>.json`.
- Dashboard:** crear/editar `frontend/src/pages/<Unidad>Dashboard.tsx` reutilizando componentes (KpiCard, Gauge, PnlMatrix, BalanceSheet, charts).
- Menú:** agregar la tarjeta en `MainMenu.tsx` (id, label, color, descripción).
- Despacho de carga:** enrutar el archivo en `/upload-informes` por nombre.
- Compilar (`npm run build`), verificar y desplegar.

10 · Convenciones y reglas críticas

- Trabajar en **español**; nombres de columnas suelen venir del Excel (con espacios) → siempre citar con comillas dobles en SQL.
- Antes de cerrar un cambio de front: `npm run build` (compila y valida TypeScript).
- Escrituras a prod: script reutilizable en `scripts/`, **dry-run primero**, y correr con `.venv/Scripts/python.exe`.
- Cargar cada informe **bajo el mes/período que representa**, no el mes de carga (evita que el dashboard muestre datos «en el futuro» o pegados).
- Para editar `.xlsx` preservando gráficos/pivots, usar **Excel COM (pywin32)**, no openpyxl.

PARTE II

Manual de usuario

Esta parte está pensada para quien **usa** la aplicación (directores y equipo). No requiere conocimientos técnicos.

A • Ingreso a la aplicación

- Tu **usuario es tu correo** (por ejemplo `nombre@sanvest.cl`).
- Puedes escribirlo en **mayúsculas o minúsculas**, da lo mismo — el sistema lo entiende igual.
- Si la contraseña no es correcta, aparece «Usuario o contraseña incorrectos». Si en cambio dice que «el servidor no respondió», no es tu clave: reintenta.
- La sesión dura unas horas; luego pedirá volver a entrar.

B • Menú principal

Tras entrar verás un saludo y luego el menú con las **tarjetas de unidades** que tienes habilitadas (cada una con su color). «Estados Financieros» (el consolidado del Grupo) aparece destacado arriba. Haz clic en *Abrir* para entrar a una unidad. Solo ves las unidades que te asignaron.

C • Cómo leer los dashboards

Elemento	Qué es
Filtros de Año / Mes	Arriba del dashboard. Eligen el período; todo el tablero se actualiza. Por defecto abre en el último mes con datos reales .
Tarjetas (KPI)	Números clave del período (ingresos, NOI, ocupación, deuda...).
Gauge (medidor)	El semicírculo de ocupación : % ocupado y m².
Combos (barra + línea)	Barra = Real, línea = Presupuesto. Muestran los últimos 12 meses que terminan en el período elegido.
Cuadros / tablas	P&L, arriendos, ventas, morosidad. Muchas filas son colapsables (haz clic en la fila con ► para desplegar el detalle).
Mes vs YTD	Casi todos los cuadros muestran el Mes y el acumulado del año (YTD) lado a lado.

D • Recorrido por unidad

DV — Desarrollo para la Venta

Avance de ventas (% y unidades), evolución de costos, y la deuda por proyecto con el capital aportado por los socios.

RR — Renta Residencial

Resultado de los edificios LAR (SOHO/PARK), ocupación del mes y del año, y KPIs por edificio (calculados en vivo).

Hotel — OLÁ

Indicadores del hotel, ocupación mes y YTD, y la deuda asociada.

USA

Resultado operativo y KPIs de Bemiston, Mila y St Grand (EE.UU.), con Real vs Presupuesto.

ICEMM — Construcción

Informe de gestión con proyección de cierre de año (FY), lo que va del año (YTD) y lo que resta (YTG), y el flujo de caja.

Atémpora — Civitas

Resultado del edificio, ocupación total, cuadro de arriendos por cliente, ventas y morosidad por tramos de mora.

Grupo — Estados Financieros

El **Balance** (réplica exacta del Excel, con las notas al pasar el mouse) y el **Estado de Resultados** consolidado del Grupo. Se puede abrir a pantalla completa y exportar. Las cuentas de detalle en cero se ocultan dejando el grupo padre visible.

E · Glosario de indicadores

Término	Significado
UF	Unidad de Fomento (moneda indexada chilena). Muchos montos van en UF.
Real	Lo efectivamente ocurrido (ejecutado).
Ppto (Presupuesto)	Lo planificado para ese período.
YTD	<i>Year To Date</i> : acumulado desde enero hasta el mes elegido.
YTG	<i>Year To Go</i> : lo que resta del año (proyección).
FY	<i>Full Year</i> : proyección del cierre del año completo (YTD + YTG).
NOI	<i>Net Operating Income</i> : resultado operativo (Ingresos – Gastos operacionales), antes de intereses y otros.
Ocupación	% de superficie (o unidades) ocupada respecto del total disponible.
Morosidad	Deuda de clientes segmentada por tramos de días de atraso (0–30, 30–60, 60–90, 90+).
Balance	Estado de situación: Activos = Pasivos + Patrimonio a una fecha.
EERR	Estado de Resultados: ingresos y gastos de un período.

F · Funciones útiles

- **Cambiar contraseña:** desde tu menú de usuario. Pide la clave actual y la nueva (mínimo 8 caracteres).
- **Comentarios:** cada unidad tiene un foro donde el directorio puede dejar observaciones, visibles para quienes ven esa unidad.
- **Exportar a PDF:** los dashboards se pueden imprimir/exportar con sus etiquetas.
- **PPT del Directorio:** visor de presentaciones/PDF en línea, por unidad.
- **Pantalla completa:** el Balance y el EERR tienen un botón para verlos a todo el ancho.

G · Para administradores

El rol **admin** ve todas las unidades y además tiene el panel de administración:

- **Cargar informes:** subir los Excel/informes por la web; el sistema los procesa y actualiza los dashboards. Cada archivo va a su unidad según el nombre.
- **Usuarios:** crear/editar usuarios, asignar rol y unidades visibles, resetear clave, activar/desactivar.
- **Accesos:** bitácora de quién entró y a qué unidades, con métricas por usuario, unidad y día.
- **Auditoría:** chequeos automáticos de calidad de datos (datos viejos, huecos, valores pegados, tablas vacías) para detectar problemas antes que el directorio.

Regla de oro al cargar: sube cada informe correspondiente al **período que representa** (el mes/trimestre del dato), no importa la fecha en que lo subes. Así los dashboards quedan cuadrados y sin datos «pegados».